MCLENNAN COMMUNITY COLLEGE · PROFESSIONAL DEVELOPMENT

You can build your own software.

Three hours. Four things you build. Nothing you have to pay for.

Ted Gonzalez, JD/MBA

You can build your own software.

Not "learn to code." Build. Describe what you need in plain English, get a working tool, put it on the internet, and keep it from leaking.

The bottleneck moved.

Syntax used to be the wall, and it kept almost everyone out. It is not the wall anymore. The wall now is knowing what to build — and knowing what will go wrong.

WHAT THE JOB ACTUALLY IS

Three steps, and only one of them is new.

01

Describe

"A form where staff request a room, and I see every request in one list." Ordinary language, no jargon.

02

Iterate

It writes it and shows you. You say what is wrong. It changes. That loop is the entire job.

03

Own it

It is real software now, on the real internet, possibly holding real data. This is the part nobody teaches.

The skill is no longer typing code. It is judgment about what you have made.

PART ONE

What you actually need

Two roads. One needs nothing but a browser. Start there — everyone does.

THE TWO ROADS

Browser, or your own machine.

ROAD ONE · TODAY

The browser builders

Open a tab, type what you want, a working app appears — and they publish it for you. Nothing to install, which matters if your laptop blocks installs.

ROAD TWO · LATER

Agents on your machine

Claude Code and its peers read and write real files and run real commands. Far more capable — and it needs installing, a paid plan and some terminal comfort.

FREE TIERS AS OF AUGUST 2026 — RE-CHECK BEFORE YOU TEACH

The builders, honestly compared.

01

Bolt.new — 300K tokens a day

The most free headroom by far. Publishes to a free address, and shows a real file tree.

02

Lovable — five messages a day

The best ten minutes you can show anyone — and five messages a day before a hard stop.

03

Vercel v0 — seven messages a day

Genuinely good visual editing for non-coders. Same daily-cap problem.

04

Replit — one published project

Most powerful and most intimidating. Database included on the free tier.

TASK 1

Ship something in thirty minutes.

1. Pick one small, annoying, real thing from your job. Not a grand idea.
2. Open bolt.new. Describe it: who uses it, and what they see.
3. Reject the first version. Name one thing that is wrong. Then another.
4. Deploy. Copy the URL. Hand it to your neighbour and watch them use it.

30 MINUTES · YOU LEAVE THIS STEP WITH A WORKING LINK

PART TWO

Your laptop is already a server

What localhost means, and why your director cannot open your link.

THE MENTAL MODEL

It is running on the machine in front of you.

Every website you have used lives on someone else's computer. Running something locally means your browser is fetching it from this same laptop. Nothing crosses the internet at all.

localhost is your office phone extension. From inside the building you dial 3000 and it rings. Nobody outside can dial your extension directly.

THE CONSEQUENCE

Which is why your link is broken for everyone else.

She types the same address and reaches her own machine, which is running nothing. 127.0.0.1 means "me" on every computer on earth. There is no version of it that means your laptop.

NOT A BUG

Nothing is broken

The address is non-routable by design.

NOT PERMISSIONS

No setting fixes it

The dev server is not listening on the network.

THE FIX

Deploy it

A shareable address means somewhere public. Part Three.

IF YOU GO LOCAL

Three installs, in this order.

01

Node.js 24 LTS

Version 24.19.0 as of August 2026. Take the LTS, not the newest. This is the engine that runs the code.

02

Git

Track Changes for a whole folder, with an undo that reaches back months. Add GitHub Desktop if you would rather click than type.

03

VS Code

Where you look at the files. You will spend less time here than you expect — the agent does most of the typing.

Before you plan on any of this: check whether your college laptop allows installs at all. If it does not, the browser tools are not the easy path — they are the only path.

PART THREE

Getting it on the internet, for free

Real free tiers. Four of them. Four completely different edges that cut.

THE CONCEPT

A static site is a folder of finished files.

WHY IT IS FREE

Nothing thinks per visitor

A printed handout in a rack by the door. Anyone can take one, the rack does no work, and ten thousand copies cost almost nothing.

WHEN YOU OUTGROW IT

The moment anything differs

Users logging in. A form whose contents must be stored. Anything you must keep secret from the visitor.

WHAT COMES NEXT

A librarian on duty

Someone looks up your record and prints you a custom page. That requires staffing, and staffing costs money.

THE FOUR FREE TIERS

Each one has an edge that cuts.

01

Cloudflare Pages — unmetered bandwidth

Best free deal — but 100,000 function requests is a DAILY bucket, resetting at midnight UTC.

02

GitHub Pages — simplest, static only

Always public. No password protection below Enterprise. No commercial transactions or SaaS.

03

Netlify — 300 credits a month

Run out and your site goes offline until the cycle resets. You cannot buy more.

04

Vercel — commercial use banned on Hobby

By its own wording that includes a paid employee writing the code.

TASK 2

Get a real address.

1 Push what you built to a GitHub repository. GitHub Desktop is fine.

2 Connect it to Cloudflare Pages. Accept every default.

3 Wait ninety seconds. You now have a public address anyone can reach.

4 Sit with that for a moment. Anyone. That is the next section.

20 MINUTES · YOU LEAVE THIS STEP LIVE ON THE INTERNET

PART FOUR

Safeguards

Why you gate a prototype, and what a web page can never keep secret.

There is no such thing as a secret in a web page.

Not minified. Not obfuscated. Not base64-encoded. Not fetched from a file with an unguessable name. Right-click, View Source.

THREE CONSEQUENCES

What that actually means for you.

01

Any key you ship is published

If it reached the browser, treat it as compromised and rotate it. Deleting does nothing.

02

Hiding a button hides nothing

A courtesy to polite users. Disabling a field stops the field, not the request.

03

So publish only what is safe to publish

Not a limitation — the design. It is why the next section matters most.

TWO WORDS PEOPLE USE INTERCHANGEABLY

Logging in is not the same as being limited.

AUTHENTICATION

Who are you?

The ID check at the door. Passwords, magic links, SSO. This part almost always works.

AUTHORIZATION

What may you see?

Which rooms the badge opens. This is the part missing in nearly every breach you will read about.

In almost every incident, everyone logged in perfectly — and then could read everyone else's records.

WHILE IT IS YOUNG

Put a gate on it.

A prototype's security is not "probably fine." It is unknown. A password in front of the whole site turns an unknown-risk public asset into a known-audience internal one. It buys you the right to be wrong.

FREE UNDER 50 USERS

Cloudflare Access

Sits in front of everything. Users verify by emailed PIN.

FREE TO 50K USERS

Supabase Auth

App-level login. Necessary but not sufficient — something still has to authorize.

NOT AVAILABLE

GitHub Pages

There is no gate. A free GitHub Pages site is public, permanently.

TASK 3

Lock the door you just opened.

1. Put Cloudflare Access in front of the site you deployed in Task 2.
2. Add your own email and the email of the person next to you.
3. Open it in a private window. Confirm you are stopped.
4. Have someone not on the list try. Confirm they are stopped too.

15 MINUTES

PART FIVE

Databases

The most important thirty minutes of the day.

THE THING THAT SURPRISES EVERYONE

The browser talks to the database directly.

Your site does not ask a server which asks the database. The visitor's own browser asks the database. That is what lets a free static site behave like a real app — and it is why one setting is not optional.

SAFE TO PUBLISH

The publishable key

Not a password — a return address saying which project you mean.

NEVER IN A BROWSER

The secret key

It carries BYPASSRLS. Every rule you write simply does not apply to it.

RLS off.

Anyone with the key — which is in your page source — can read every row. They can also insert, update and delete.

Marisol Alcaraz · Devonte Whitfield · Priya Okonkwo
Tomas Brannigan · Shanice Vasquez · + 118 more rows

RLS on.

The same key. The same request. And the database returns nothing — because the rule ran inside the database after the request arrived.

```
[ ]
```

Empty. The app cannot talk its way around it. Neither can anyone else.

WHAT IT IS

A rule that lives in the cabinet, not in the app.

A normal database permission is a key to a filing cabinet — you are either in or out. Row-level security says: you may open this cabinet, but you only ever see the folders with your name on them.

The app cannot talk its way around it, because the check happens inside the database after the request arrives.

HOW IT GOES WRONG

Four that happen constantly.

01

The table was made in SQL, not the Table Editor

Dashboard tables get RLS automatically. SQL tables do not — and SQL is what an AI assistant writes.

02

A policy that says USING (true)

Means "everyone, always." Passes every checkbox, protects nothing. It looks like security.

03

The table you added last month

Month one was locked down. Month three was not. Nobody re-audits.

04

The secret key ended up in the browser

Someone hit a permissions error and swapped keys. It works instantly — which is why it is seductive.

HOW IT GOES WRONG

And three that look fine right up until they do not.

05 **A security-definer view**

Runs as its creator, not the caller. RLS is on, the policy is right, data leaks anyway.

06 **RLS on with no policy — then panic**

Blocks everything, which is correct. Looks broken. The fix people reach for is turning RLS off.

07 **USING without WITH CHECK**

It matters when the rows you may read are broader than those you may write. Omit it and Postgres reuses USING.

THIS HAS ALREADY HAPPENED

Repeatedly, and recently.

10.3%

of projects scanned

170 of 1,645 Lovable projects were exposing data in one researcher's 2025 scan.

CVSS 9.3

CVE-2025-48757

Unauthenticated read or write to arbitrary tables of generated sites, per the GitHub advisory.

175

instances of real data

Medical records, bank details, phone numbers — found across vibe-coded apps.

A01

on the OWASP Top 10

Broken access control is the number one risk in the industry's canonical list.

GHSA-773x-pxjg-gxgx · Matt Palmer disclosure statement · OWASP Top 10:2025

A tool telling you that you are secure is not the same as being secure.

When the platform responded, it shipped a scanner that checked whether a policy existed — not whether it was correct. So USING (true) passed the audit with a green checkmark.

TASK 4

Break your neighbour's app.

1. Add a database with two or three invented records. Fake. Invent them.
2. Turn RLS on and write one policy so a user sees only their own rows.
3. Open Database → Security Advisor. Fix everything. Re-run until clean.
4. Swap laptops. Your job is to see data you should not. Find the key.
5. Whatever you find, tell them. That is the exercise.

25 MINUTES · NOBODY'S SURVIVES THE FIRST TIME

PART SIX

If you want to sell it

Start with the question almost nobody asks.

MCC POLICY E-XVII, EFFECTIVE FEBRUARY 2011

Who owns what you build?

YOURS

Your own time, your own resources

The policy gives employees exclusive rights to what they develop on their own time without substantial institutional resources.

JOINTLY OWNED

Substantial institutional resources

Anything developed using them is jointly owned by the College and you. "Substantial" is nowhere defined in the policy.

The policy predates cloud services, institutional software licences and AI tools entirely. Expect ambiguity — do not expect it to resolve in your favour.

TWO THINGS PEOPLE GET WRONG

"Jointly owned" is not a compromise.

THE IP TRAP

It means a permanent consent requirement

Joint owners each hold an undivided interest — but neither can grant an exclusive licence or sell the whole thing without the other. A jointly owned product is effectively unsellable, and revenue carries a duty to account.

THE OTHER REGIME

It is not only an IP question

Texas Penal Code § 39.02 makes it abuse of official capacity for a public servant to misuse government property or services for personal benefit, graded by value.

Personal device. Personal accounts. Nights and weekends. No institutional data, ever. And get a written determination before you build — not after you have a customer.

THEN THE ORDINARY QUESTIONS

Payments, tax, structure.

01 **Stripe is 2.9% + 30¢, and every tax obligation stays yours**

A merchant of record charges ~5% and becomes the legal seller, remitting tax for you.

02 **Texas taxes SaaS, and you live in Texas**

6.25% state plus up to 2% local, on 80% of the price. No safe harbor — it starts at sale one.

03 **An LLC is one layer, not a shield**

$300 to form. It covers contract debts — not your own torts, which is what you get sued for.

04 **The Texas privacy law has no threshold**

California's CCPA has limits a side project will never hit. The Texas act applies based on doing business here.

THE NUMBER NOBODY PUTS ON A SLIDE

Most of them earn nothing.

54%

earn zero

Of indie products with verified payment data. Not "not much" — nothing.

~5%

clear $100K a year

Measured among people already serious enough to launch publicly.

1

in every story you have heard

Every "$20K a month solo founder" post is a draw from the far tail, amplified because it is rare.

Stripe-verified Indie Hackers data, 2022 — dated, and subject to survivorship bias

Treat it as skill acquisition with a lottery ticket attached.

The skills transfer to your day job with certainty. The revenue does not. Do the IP analysis before you write code — it is free, and it is the only irreversible step.

BEFORE YOU MAKE ANYTHING PUBLIC

The gate.

01 **I know exactly what data this holds**

And whether any is about a real person. Student data means I talked to IT.

02 **RLS is on for every single table**

Not most. Every one. And Security Advisor is clean after my last change.

03 **No secret key anywhere the browser can reach**

And anything ever committed or deployed has been rotated, not deleted.

04 **It is gated if it only needs thirty colleagues**

And two-factor is on for the database, code host and hosting account.

05 **There is a named owner who is not only me**

Plus a weekly reminder so the database does not pause, and one export saved.

If you cannot explain what stops a stranger from reading your database, it is not ready.

Assume everything in it becomes public tomorrow. Are you comfortable? If not, the answer is not better security. It is different data.